

SUPPLEMENTARY REFERENCE

AgentCore Memory — Gaps, Extensions & 2026 Research

FileSessionManager · Conversation Managers · Custom Strategy Wiring Structured Extraction · Graph Memory · Emerging Research 2025-2026

Topic	Content in this document
FileSessionManager	When to use, backends, vs AgentCore Memory, wiring code
Conversation Managers	Sliding window, summarising, null — config & prompt customisation
Custom Strategy Wiring	Built-in overrides, self-managed Lambda pipeline, SNS/S3 trigger
Structured Extraction	Entity schema, Pydantic wiring, prompt changes, namespace design
Graph Memory	Graphiti/Zep, MAGMA, integration patterns with AgentCore
2025-2026 Research	MemOS, Nemori, sleep-time compute, MAGMA, HyperGraphRAG, Hindsight

Supplement to: AWS AgentCore Memory Architecture Guide v2.0 (April 2026)

1. FileSessionManager — The Missing Chapter

1.1 What the Source Document Omitted

The Architecture Guide covers AgentCore Memory exhaustively but makes no mention of the Strands SDK's own built-in session persistence layer: **FileSessionManager** and **S3SessionManager**. These are *not* the same as AgentCore Memory. They are lighter-weight mechanisms that persist the raw conversation message list to disk or S3 so an agent can resume exactly where it left off after a process restart, without incurring any AgentCore API costs.

1.2 Decision Matrix — FileSessionManager vs AgentCore Memory

Criterion	FileSessionManager	AgentCore Memory
What is stored	Full raw message list (JSON files per message)	Events → extracted long-term memories (vector store)
Retrieval	Chronological replay — all messages restored	Semantic search — top-K relevant memories
Cost	Storage cost only (negligible / S3 prices)	Per API call + storage + consolidation job
Latency	Local: <5 ms S3: ~50 ms cold	~200 ms p99 (semantic search)
Extraction / Summarisation	None — raw turns only	Yes — managed or custom strategy
Cross-session intelligence	No — same session ID required	Yes — actor_id links sessions
GDPR compliance burden	Operator-owned encryption + lifecycle	Managed; CMK; erasure API built-in
Best for	Local dev, PoC, single-user resumption after restart	Production; personalisation; multi-session continuity
Conflict with AgentCore?	Can be used together — orthogonal layers	N/A

1.3 When to Use FileSessionManager

Use FileSessionManager when: (1) You want zero infrastructure and zero API cost for conversation continuity. (2) Your use case is a single-user CLI tool, desktop app, or developer workflow where a session must resume after a restart but cross-session semantic recall is not required. (3) You are building a Proof-of-Concept and want to iterate quickly without provisioning AgentCore. (4) You run **BidiAgent** (bidirectional streaming / voice) — FileSessionManager handles the reconnect-after-timeout pattern natively for that agent class.

Do NOT use FileSessionManager as a substitute for AgentCore Memory when: You need semantic search over historical context, user preference extraction, GDPR right-to-erasure by actor_id, multi-session recall across days or weeks, or multi-agent shared memory.

1.4 Backends Available

Backend	Import	Storage Path	When to Use
FileSessionManager	strands.session.file_session_manager	./sessions/ (default: /tmp/strands/sessions)	Local dev, PoC, desktop agents
S3SessionManager	strands.session.s3_session_manager	s3://bucket/prefix/session_/	Production; serverless Lambda agents
RepositorySessionManager	strands.session.repository_session_manager	Custom backend (DynamoDB, RDS, etc.)	Enterprise; custom compliance requirements

1.5 Wiring Code — Key Patterns

Single agent (file backend):

```
from strands import Agent
from strands.session.file_session_manager import FileSessionManager
session_manager = FileSessionManager( session_id='user-abc-123', storage_dir='./sessions' # do
```

```
NOT use /tmp in production ) agent = Agent( agent_id='support_bot', # REQUIRED when using
session manager session_manager=session_manager, tools=[...] )
```

Multi-agent constraint (critical):

WARNING You CANNOT attach a session manager to individual agents inside a multi-agent graph or swarm. Only the ORCHESTRATOR should hold the session_manager. Sub-agents must be created without one. # CORRECT orchestrator = Agent(session_manager=session_manager, ...) sub_agent_a = Agent(...) # no session_manager sub_agent_b = Agent(...) # no session_manager

S3 backend (production):

```
from strands.session.s3_session_manager import S3SessionManager session_manager =
S3SessionManager( session_id='user-abc-123', bucket='my-agent-sessions', prefix='sessions/',
region='eu-central-1' )
```

Tip: For EU banking deployments using S3SessionManager, apply S3 server-side encryption with CMK (SSE-KMS), enable versioning, and set a lifecycle rule to expire objects after your retention window. FileSessionManager has no built-in encryption — never use it with real PII.

2. Conversation Managers — Sliding Window, Summarising & Null

The Architecture Guide references 'context compaction' and the ACON framework but does not explain the three built-in ConversationManagers in the Strands SDK. These are independent of AgentCore Memory and control how the in-memory message list is trimmed before each model call.

2.1 The Three Managers

Manager	Default?	Effect on Context	Effect on Cost
SlidingWindowConversationManager	YES	Keeps the last N messages. Drops oldest when window_size exceeded. Handles dangling message cleanup and overflow trimming.	Most agents. Simple, predictable, low overhead.
SummarizingConversationManager	No	Summarises the oldest (summary_ratio) fraction of history with an LLM call. Preserves preserve_recent_messages most recent turns verbatim.	Long advisory sessions, 1h+ conversations, wealth management agents.
NullConversationManager	No	Does nothing. Full history grows unbounded.	Short single-turn queries; testing; agents where full history is required and context budget is managed externally.

2.2 SlidingWindowConversationManager — Full Config

```
from strands.agent.conversation_manager import SlidingWindowConversationManager
agent = Agent(
    conversation_manager=SlidingWindowConversationManager(
        window_size=40, # max messages to keep
        (default: varies by model) per_turn=True, # apply trim before EVERY model call # per_turn=3 #
        or apply every 3 model calls (reduces overhead) should_truncate_results=True # truncate
        oversized tool results ) )
```

When sliding window alone is used with AgentCore Memory, the pattern is: AgentCore long-term memories (injected via MemoryRetrievalHook into the system prompt) survive the window trim because they live in the system prompt, not the message list. Only raw conversation turns are dropped. This is the preferred pattern for most production agents.

2.3 SummarizingConversationManager — Config & Prompt Customisation

```
from strands.agent.conversation_manager import SummarizingConversationManager
BANKING_SUMMARY_PROMPT = ''' You are summarising a financial advisory conversation. Create a
concise summary that: - Preserves all stated financial goals, risk tolerance, and product
preferences - Retains specific amounts, account types, and regulatory mentions (MiFID II, GDPR)
- Omits pleasantries and conversational filler - Flags any pending actions the client requested
Format as structured bullet points. '''
agent = Agent(
    conversation_manager=SummarizingConversationManager(
        summary_ratio=0.3, # summarise oldest 30%
        when reducing preserve_recent_messages=10, # always keep last 10 turns verbatim
        summary_prompt=BANKING_SUMMARY_PROMPT # domain-specific prompt ) )
```

2.4 Interaction Matrix — Conversation Manager vs AgentCore Memory

Combination	Behavior	Recommendation
SlidingWindow + AgentCore short-term	Raw turns trimmed in RAM; full event log persisted to AgentCore. No conflict.	YES — default production pattern
SlidingWindow + AgentCore long-term retrieval	Long-term memories injected into system prompt (survive window). Window trims raw turns only.	YES — best for relationship agents
Summarising + AgentCore SUMMARIZATION strategy	Double summarisation. AgentCore already summarises post-session. Redundant cost.	AVOID — pick one or the other
Summarising + AgentCore USER_PREFERENCE	Summariser may drop preference signals before AgentCore extraction fires. Risk of missed preferences.	CAUTION — lower summary_ratio; set AgentCore trigger earlier

Null + AgentCore Memory	Full history grows unbounded in RAM. AgentCore handles persistence. Works for short sessions.	Only for short sessions (<50 turns)
-------------------------	---	-------------------------------------

3. Custom Strategy Wiring — Built-in Overrides & Self-Managed Lambda

The Architecture Guide mentions 'self-managed strategy Lambda' but does not explain the three-tier strategy system or the infrastructure wiring required. This section fills that gap.

3.1 Three Strategy Tiers

Tier	AgentCore Configuration	Customisation Level	Extra Infrastructure
Built-in	AgentCore service account (fully managed)	Zero — fixed algorithms	None
Built-in Override (Custom Prompt)	AgentCore service account; your prompt appended	Medium — prompt append only; model choice	Bedrock model access in your account
Self-Managed	Your Lambda / pipeline (you own everything)	Full — any model, any schema, any logic	S3 bucket + SNS topic + IAM role + Lambda

3.2 Built-in Override — Prompt Customisation (No Lambda Needed)

Built-in overrides let you append domain-specific instructions to AgentCore's managed extraction prompt. This is the right choice when the built-in schema (fact / preference / summary) is sufficient but the extraction misses domain terms.

```
import boto3 client = boto3.client('bedrock-agentcore-control') client.update_memory(
memoryId='my-banking-memory', memoryStrategies=[ { 'semanticMemoryStrategy': { 'name':
'BankingSemanticMemory', 'configuration': { 'semanticOverrideConfiguration': {
'extractionConfiguration': { 'appendToPrompt': ''' Pay special attention to: - Financial
products: ISAs, SIPPs, GIAs, bonds, ETFs - Risk appetite: cautious, balanced, adventurous -
Regulatory signals: MiFID II suitability, KYC flags - Life events: retirement horizon,
inheritance, divorce Extract these as facts even if expressed informally. ''', 'modelId':
'anthropic.claude-sonnet-4-5' } } } } ] )
```

3.3 Self-Managed Strategy — Full Infrastructure Wiring

Required infrastructure (Terraform / CDK):

Resource	Purpose	Key Config
S3 bucket (payload delivery)	AgentCore drops batched event payloads here as JSON	Lifecycle: delete after 7 days; SSE-KMS
SNS topic (notification)	AgentCore publishes job-start notification	FIFO if ordering matters; SQS subscription for Lambda
IAM role (AgentCore trust)	AgentCore assumes this role to write S3 + publish SNS	Trust: bedrock-agentcore.amazonaws.com
Lambda function (extractor)	Your custom extraction + consolidation logic	Timeout: 5 min; mem: 1 GB; VPC if needed
EventBridge / SQS (trigger)	Routes SNS notification to Lambda	DLQ mandatory; retry=2; visibility 300s

Self-managed trigger configuration in CreateMemory:

```
client.create_memory( name='banking-self-managed', memoryStrategies=[ {
'customMemoryStrategy': { 'name': 'FinancialEntityExtractor', 'configuration': {
'customConfiguration': { 'lambdaArn': 'arn:aws:lambda:eu-central-1:123:function:extractor',
'triggerConfiguration': { 'messageCountThreshold': 10, # trigger after 10 events
'idleTimeoutSeconds': 1800, # or 30 min idle 'tokenCountThreshold': 4000 # or 4K tokens },
'deliveryConfiguration': { 's3BucketName': 'my-payload-bucket', 'snsTopicArn':
'arn:aws:sns:eu-central-1:123:memory-jobs' } } } } ] )
```

4. Structured Extraction — Schema, Wiring & Prompt Changes

Structured extraction transforms free-form conversation into typed, queryable memory records. The Architecture Guide mentions it for 'KYC facts, product knowledge, org hierarchy' but omits the exact schema, wiring, and prompt engineering required.

4.1 When Structured Extraction Is Needed

Signal	Example	Extraction Type
Domain-specific entities	ISIN, LEI, BIC, product code, risk rating	Self-managed Lambda with custom schema
Relational facts that change	Account manager changed from Alice to Bob	Semantic + temporal tracking (consider Graphiti)
Typed numerical fields	Gross income: £85,000 / Risk score: 7/10	Self-managed Lambda; Pydantic schema
Multi-entity conversations	Joint mortgage with two applicants	Self-managed; namespace per applicant
Compliance-mandatory attributes	KYC category, PEP status, AML flag	Self-managed; immutable record; WORM
Standard preferences	Preferred language, preferred channel	Built-in USER_PREFERENCE — no custom wiring
Session summaries	What happened in this support call	Built-in SUMMARIZATION — no custom wiring

4.2 Entity Schema — Pydantic Model (Banking Example)

```
from pydantic import BaseModel, Field from typing import Optional, List from enum import Enum
class RiskAppetite(str, Enum): CAUTIOUS = 'cautious' BALANCED = 'balanced' ADVENTUROUS =
'adventurous' class FinancialProfile(BaseModel): annual_income_gbp: Optional[int] =
Field(None, description='Stated gross annual income') risk_appetite: Optional[RiskAppetite] =
Field(None) investment_horizon_yr: Optional[int] = Field(None, description='Years to target
event') product_interests: List[str] = Field(default_factory=list, description='ISA, SIPP,
GIA, bond, ETF...') life_events: List[str] = Field(default_factory=list,
description='retirement, inheritance, divorce') kyc_category: Optional[str] = Field(None)
pep_status: bool = Field(False) EXTRACTION_PROMPT = f''' Extract a FinancialProfile JSON from
the conversation below. Schema: {FinancialProfile.model_json_schema()} Rules: - Only include
fields explicitly stated or strongly implied - Set pep_status=true only if PEP is explicitly
mentioned - Return {} if no financial profile data found - NEVER invent values not in the
conversation Return only valid JSON matching the schema. No explanation. '''
```

4.3 Lambda Extractor Skeleton

```
import json, boto3 from pydantic import ValidationError bedrock =
boto3.client('bedrock-runtime', region_name='eu-central-1') agentcore =
boto3.client('bedrock-agentcore', region_name='eu-central-1') def handler(event, context): #
1. Get payload from S3 s3 = boto3.client('s3') payload = json.loads(s3.get_object(
Bucket=event['payloadBucket'], Key=event['payloadKey'])['Body'].read()) conversation =
payload['events'] # list of {role, content} actor_id = payload['actorId'] memory_id =
payload['memoryId'] # 2. Call Claude for structured extraction response = bedrock.invoke_model(
modelId='anthropic.claude-sonnet-4-5', body=json.dumps({'messages': [ {'role':'user',
'content': EXTRACTION_PROMPT + json.dumps(conversation)} ], 'max_tokens': 1000}) ) profile =
FinancialProfile.model_validate_json(
json.loads(response['body'].read())['content'][0]['text']) # 3. Batch write to AgentCore
Memory records = [] for field, value in profile.model_dump(exclude_none=True).items():
records.append({'content': {'text': f'{field}: {value}'}, 'namespace':
f'users/{actor_id}/financial-profile', 'memoryRecordId': f'{actor_id}-{field}' }) if records:
agentcore.batch_create_memory_records( memoryId=memory_id, records=records )
```

4.4 System Prompt Changes Required

When structured extraction memories are retrieved and injected into the system prompt, you must change how the agent interprets and uses them. Add the following block to your system prompt:

```
## Structured Memory Context The following structured facts have been retrieved from long-term
memory for this customer. These are AUTHORITATIVE – do not contradict them unless the customer
explicitly updates them. {{STRUCTURED_MEMORIES}} {# injected by MemoryRetrievalHook #} Rules:
1. If the customer states new information that contradicts a memory fact, update your response
to reflect the NEW value and call the memory_write tool to update the record. 2. Never recite
raw memory records verbatim – synthesise naturally. 3. For financial recommendations, ALWAYS
confirm structured preferences are still current: 'I have on record that your risk appetite is
balanced – is that still the case?' 4. If a structured field is present, do not ask the
customer to repeat that information.
```


5. Graph Memory — Graphiti, MAGMA & Integration Patterns

The Architecture Guide briefly mentions Zep/Graphiti as the 'best for temporal graph reasoning' but does not explain when graph memory becomes necessary, how it works, or how it integrates with AgentCore Memory. This section fills that gap.

5.1 What Graph Memory Solves

Vector stores answer 'what memories are semantically similar to this query?' Graph memory answers 'how do these entities relate, how did those relationships change over time, and what can I infer by traversing the relationship graph?'

Problem	Vector Store (Semantic Only)	Graph Memory (Relationships + Time)
Alice was PM until Jan; Bob took over	Both facts retrieved — model must infer recency	Temporal edge: Alice[valid_to=Jan], Bob[valid_from=Jan]. Query returns correct current owner.
Client holds ISIN XY001Z in ISA and GIA	Two separate facts; no link	Entity [ISIN:XY001Z] connected to [Account:ISA] and [Account:GIA] — query traverses both
Fraud pattern: same entity, 3 products, 30 days	Requires 3 separate retrieval queries + LLM join	Graph traversal: entity → transactions in time window — single query
Organisation hierarchy changed	Stale facts accumulate	Edge invalidation: old edge [valid_to=now]; new edge created — history preserved
Who approved the loan?	May retrieve multiple approval facts	Path: [Loan:L001] -[APPROVED_BY]-> [Person:Jane] with timestamp

5.2 Graphiti Architecture (Zep's Open-Source Core)

Graphiti ingests 'episodes' (raw text, JSON, or structured records) and autonomously decomposes them into: (1) **Entities** — named nodes (Person, Account, Product, Organisation). (2) **Edges** — typed relationships with validity windows (t_valid, t_invalid). (3) **Episode nodes** — ground truth provenance tracing every fact to its source. Retrieval uses a triple-modality hybrid: semantic embeddings + BM25 keyword search + graph traversal — no LLM calls during retrieval, achieving P95 latency of ~300ms.

5.3 MAGMA — Multi-Graph Architecture (2026)

MAGMA (Multi-Graph based Agentic Memory Architecture, arXiv 2601.03236, Jan 2026) represents the current state-of-the-art research direction. It decomposes memory into four orthogonal graph layers:

Graph Layer	Stores	Example Query
Semantic graph	Entity facts and properties	What products does Alice hold?
Temporal graph	Fact validity windows; event sequences	Who was the account manager in Q3 2024?
Causal graph	Cause-effect relationships between events	Why was the limit reduced? (causal chain)
Entity graph	Cross-entity relationships and hierarchies	Who reports to the compliance officer?

MAGMA achieves an LLM-as-judge score of 0.70 on the LoCoMo benchmark — the highest reported in peer-reviewed evaluation as of Q1 2026, outperforming Graphiti/Zep, A-MEM, MemoryOS, and Nemori by 18-45% relative margin.

5.4 Integration Pattern — Graphiti + AgentCore Memory

Graphiti and AgentCore Memory are complementary, not competing. The recommended pattern for EU banking agents requiring temporal relationship tracking:

Layer	System	Stores	Retrieved By
Short-term events	AgentCore Memory	Raw conversation turns (7-90d)	AgentCoreMemorySessionManager
Long-term preferences	AgentCore Memory	Summarised prefs, key facts	MemoryRetrievalHook (semantic)

Temporal entity graph	Graphiti (self-hosted Neo4j)	Entity relationships + validity windows	Graphiti search API (hybrid)
Audit ledger	AgentCore (Transaction pattern) + S3 WORM	Immutable event log	CloudTrail / S3 query

Graphiti wiring in a Strands hook:

```
from graphiti_core import Graphiti class GraphitiRetrievalHook: def __init__(self, graphiti: Graphiti, actor_id: str): self.graphiti = graphiti self.actor_id = actor_id def on_message_added(self, event): if event.message.role == 'user': # Temporal-aware retrieval graph_facts = self.graphiti.search( query=event.message.content, center_node_uuid=self.actor_id, num_results=5 ) # Inject alongside AgentCore memories graph_context = self._format(graph_facts) event.agent.system_prompt = ( graph_context + '\n\n' + event.agent.system_prompt ) # Register both hooks agent = Agent( hooks=[ PIIRedactionHook(redactor), MemoryRetrievalHook(agentcore_client, memory_id, actor_id), GraphitiRetrievalHook(graphiti_client, actor_id), ConsentCheckHook(consent_service), MemoryPersistenceHook(session_manager), ] )
```

EU Banking Note: Self-hosting Graphiti requires Neo4j Enterprise for production (AuraDB available in eu-west-1). Apply AES-256 encryption at rest, TLS 1.3 in transit, and ensure all graph data stays within the EU. Graphiti does not provide a managed GDPR right-to-erasure — build a custom node-deletion workflow triggered by the memory_delete skill.

6. Emerging Research & Adoptions — Agent Memory 2025–2026

6.1 Research Taxonomy

The field of agent memory has matured significantly since the Architecture Guide's reference points. The ICLR 2026 MemAgents Workshop marked the field's academic coming-of-age. Key research directions as of April 2026:

System / Paper	Year	Key Contribution	Production Potential
Zep / Graphiti (arXiv 2501.13956)	2025	Temporal knowledge graph with bitemporal edges. 94.8% on DMR benchmark. Hybrid retrieval: semantic + BM25 + graph traversal. P95 < 300ms.	HIGH — GA product; used in CRM, compliance, medical agents. Best temporal reasoning available.
MAGMA (arXiv 2601.03236)	Jan 2026	Four-layer graph architecture (semantic / temporal / causal / entity). State-of-the-art 0.70 LoCoMo score. Policy-guided retrieval traversal.	MEDIUM — research; no production SDK yet. Watch for OSS release in H2 2026.
MemOS / MemoryOS (EMNLP 2025 Oral)	2025	OS-inspired hierarchical memory manager. Global, local, and working memory buffers. Semantic-focused storage policies.	MEDIUM — conceptual influence on AgentCore tiered design. No direct integration.
Nemori (arXiv 2025)	2025	Cognitive-science-inspired self-organising memory. Agents construct their own memory via reinforcement-like signals.	LOW-MEDIUM — research direction for adaptive agents. Not production-ready.
Letta Sleep-Time Compute (2025)	2025	Background async consolidation during agent 'sleep'. Anticipatory pre-computation: predicts likely future queries and pre-fetches context.	MEDIUM — concept adopted partially by AgentCore's EventBridge async consolidation. Watch Letta 2.0.
Hindsight (2025)	2025	Four-network architecture: facts, experiences, opinions, observations. Confidence-scored beliefs that update with evidence. 89.61% LoCoMo, 91.4% LongMemEval.	MEDIUM — opinion network concept is novel for agents that need to track user sentiment over time.
HyperGraphRAG (arXiv 2025)	2025	Hyperedges connecting 3+ entities simultaneously. Captures complex multi-entity relationships flat graphs miss (e.g., joint accounts, consortium loans).	LOW-MEDIUM — promising for complex banking entity relationships. Research stage.
Mem0 v3 — Context Engineering (2025)	2025	Repositioned from 'memory API' to 'context engineering platform'. Three-tier: user / session / agent scopes. Self-edit on conflict (no duplicate accumulation).	HIGH — 48K GitHub stars; active production use. LongMemEval: 49% (vs Zep's 63.8%).
Amazon S3 Vectors (GA re:Invent 2025)	Dec 2025	Native vector storage in S3. Billion-vector scale. Purpose-built for AI agents; subsecond latency for frequent queries. AWS-native alternative to managed vector DBs.	HIGH — AWS-native; integrates with Strands S3SessionManager pattern for hybrid memory.
Cognee (2025-2026)	2025	Poly-store: Neo4j/FalkorDB + SQLite/Postgres + vector. Background Memify Pipeline enriches existing knowledge. Fully local deployment for air-gapped environments.	MEDIUM — best for data-residency-strict environments where both graph and local deployment are required.

6.2 Benchmark Comparison (April 2026)

System	DMR	LoCoMo (0.00-1.00)	LongMemEval	Temporal Latency
Graphiti / Zep	94.8%	~0.65	63.8%	P95 ~300ms
MAGMA (research)	—	0.70 (highest)	—	Not published
Hindsight	—	—	91.4%	Not published
Mem0 (managed)	—	—	49.0%	7-8s at scale
AgentCore Memory	—	—	—	~200ms p99
Letta / MemGPT	93.4%	~0.50	—	Varies (file traversal)

Note: — indicates not publicly evaluated on that benchmark as of April 2026.

6.3 What to Watch in H2 2026

Development	Why it Matters for EU Banking Architecture
MAGMA open-source release	If OSS, could complement AgentCore with causal graph layer for AML pattern detection
Zep Cloud EU region expansion	Currently limited EU region availability; Frankfurt expansion would unlock GDPR-compliant temporal graph
Amazon S3 Vectors + AgentCore integration	Native AWS vector store could replace internal AgentCore vector index; watch for announced integration
Letta 2.0 sleep-time compute GA	If Strands SDK adopts sleep-time pattern, could replace EventBridge consolidation with smarter pre-fetching
LoCoMo-Plus benchmark adoption	New benchmark shows all systems degrade on cue-trigger semantic disconnect — re-evaluate vendor claims against LoCoMo-Plus not original LoCoMo
EU AI Act Article 13 transparency rules	Memory systems influencing high-risk AI outputs may require new disclosure obligations effective 2026

7. Consolidated Decision Guide — Which Memory Layer for Which Problem

7.1 Complete Memory Layer Selection Matrix

Requirement	FileSession Manager	Conversational Manager	AgentCore Short-Term	AgentCore Long-Term	Graphiti / Graph
Resume after process restart	YES — primary use	No	Partial (events persist)	No	No
Trim tokens within session	No	YES — primary use	No	No	No
Domain-specific summarisation	No	YES (custom prompt)	No	YES (strategy override)	No
Cross-session preference recall	No	No	No	YES	No
Temporal entity tracking	No	No	No	Limited	YES — primary use
Right-to-erasure (GDPR Art.17)	Manual S3 delete	In-memory only	YES (delete-event API)	YES (namespace delete)	Manual Neo4j delete
Multi-agent shared context	Orchestrator only	No	YES (shared namespace)	YES (pub/sub)	YES (shared graph)
Cost vs. no memory	Near-zero	Zero (in-process)	Low (per GB)	Medium (per API call)	Medium-high (Neo4j)
Production GDPR compliance	Operator-managed	N/A	YES (managed)	YES (CMK, erasure)	Operator-managed

7.2 Five Rules Not in the Original Document

Rule	Detail
1. FileSessionManager ≠ AgentCore Memory	Use FileSessionManager for restartability. Use AgentCore for intelligence (recall, personalisation, extraction). They stack — one does not replace the other.
2. Never double-summarise	If you use SummarizingConversationManager, disable the AgentCore SUMMARIZATION strategy (and vice versa). Two summarisers create redundant cost with no quality gain.
3. Structured extraction needs a custom Lambda	Built-in strategies extract generic facts/preferences. Banking entities (ISIN, LEI, PEP, risk score) need a self-managed Lambda with a Pydantic schema and appendToPrompt or full custom extraction.
4. Graph memory is complementary, not competing	Add Graphiti when you need 'who owned X until when?' — not to replace AgentCore. Run both hooks; inject both contexts into the system prompt.
5. New benchmarks break old rankings	LoCoMo-Plus (Feb 2026) shows all systems degrade on indirect cue queries. Do not evaluate memory vendors on LoCoMo alone. Run your own domain evals.

Supplement prepared April 2026. Aligns with Strands Agents SDK 1.0, AgentCore Memory GA (Oct 2025), Graphiti/Zep current release, and peer-reviewed research to Q1 2026. Review alongside the primary Architecture Guide v2.0. Regulatory citations follow the same framework: GDPR (2018), DORA (2025), EBA ML Guidelines, MiFID II.